

LinguaLearn - Die Plattform zum interaktiven Sprachenlernen

Modul: Objektorientierte Softwareentwicklung (OOSD)

Atieh Goleij, Kiavash Mansourijamshidi

Inhaltsverzeichnis:

1. Motivation und Anforderungen

- 1.1 Projektidee
- 1.2 Funktionale Anforderungen
- 1.3 Recherche / Bestehende Lösungen

2. Planung und Entwurf

- 2.1 Technologien und Werkzeuge
- 2.2 Architekturüberblick
- 2.3 Objektorientierter Entwurf

3. Entwicklung

- 3.1 Implementierte Funktionen
- 3.2 OOP-Konzepte im Projekt
- 3.3 Technische Herausforderungen & Lösungen

4. Inbetriebnahme

- 4.1 Voraussetzungen
- 4.2 Lokale Inbetriebnahme

5. Fazit

- 5.1 Anforderungsabgleich
- 5.2 Lernerfahrungen
- 5.3 Erweiterungsmöglichkeiten

6. Quellen

1. Motivation und Anforderungen

1.1 Projektidee

Die German Learning App ist eine webbasierte Sprachlern-Anwendung, die nach dem Vorbild moderner Gamification-Plattformen konzipiert wurde. Die grundlegende Idee besteht darin, das Erlernen der deutschen Sprache durch interaktive Übungen, ein Punktesystem (XP) und Fortschrittsverfolgung motivierender zu gestalten. Anders als bei traditionellen Lernmethoden setzt die Anwendung auf kurze, fokussierte Lektionen mit sofortigem Feedback, um den Lernenden kontinuierliche Erfolgserlebnisse zu ermöglichen.

Die Anwendung richtet sich primär an Anfänger (Niveau A1) und bietet verschiedene Übungsformate: Multiple-Choice, Lückentexte, Zuordnungsübungen und Satzordnungsaufgaben.

1.2 Funktionale Anforderungen

FA-01: Benutzerregistrierung

Das System ermöglicht neuen Benutzern die Erstellung eines Kontos mit Benutzername, E-Mail-Adresse und Passwort. Die Eingaben werden validiert (eindeutige Benutzernamen und E-Mail-Adressen, Mindestlänge für Passwörter).

FA-02: Benutzeranmeldung

Registrierte Benutzer können sich mit Benutzername und Passwort anmelden. Die Authentifizierung erfolgt über Spring Security mit BCrypt-Passwortverschlüsselung.

FA-03: Kursstruktur navigieren

Das Dashboard zeigt alle verfügbaren Kurse, Units und Lektionen in einer übersichtlichen Hierarchie an. Benutzer können den Lernfortschritt visuell erkennen.

FA-04: Lektionen absolvieren

Benutzer können freigeschaltete Lektionen starten und durcharbeiten. Jede Lektion enthält mehrere Übungen verschiedener Typen.

FA-05: Übungstypen

Das System unterstützt vier Übungsformate:

- Multiple Choice: Auswahl der richtigen Antwort aus mehreren Optionen
- Lückentext (Fill Blank): Ergänzung fehlender Wörter
- Zuordnung (Match Pairs): Verbinden zusammengehöriger Begriffe
- Satzordnung (Sentence Order): Wörter in die richtige Reihenfolge bringen

FA-06: Antwortvalidierung mit Feedback

Bei jeder Übung erhält der Benutzer sofortiges Feedback (richtig/falsch), die korrekte Antwort bei Fehlern sowie eine optionale Erklärung.

FA-07: XP-System

Für korrekte Antworten werden Erfahrungspunkte (XP) vergeben. Die Basis-XP betragen 10 Punkte, mit Streak-Bonus bei aufeinanderfolgenden korrekten Antworten.

FA-08: Fortschrittsverfolgung

Der Fortschritt jedes Benutzers wird pro Lektion gespeichert, einschließlich Anzahl korrekter Antworten, Gesamtantworten, beste Punktzahl und Abschlussstatus.

FA-09: Freischaltlogik

Lektionen werden sequentiell freigeschaltet. Eine Lektion gilt als bestanden bei mindestens 80% korrekten Antworten, woraufhin die nächste Lektion zugänglich wird.

FA-10: Übungsmodus

Bereits abgeschlossene Lektionen können erneut durchgearbeitet werden (Practice Mode), jedoch ohne erneute XP-Vergabe.

FA-11: Benutzerprofil

Eine Profilseite zeigt Gesamtstatistiken (Total XP, Streak, abgeschlossene Lektionen) sowie die Lernhistorie an.

1.3 Recherche / Bestehende Lösungen

Bestehende Sprachlern-Plattformen wurden analysiert:

- **Duolingo:** Marktführer mit Gamification (XP, Streaks), kurzen Lektionen und vielfältigen Übungsformaten
- **Babbel:** Strukturiertes Lernen mit umfangreicheren Grammatikerklärungen
- **Memrise:** Fokus auf Spaced Repetition und community-generierte Inhalte

Aus der Analyse wurden folgende Designentscheidungen abgeleitet: XP-System und Freischaltlogik für Motivation, vier Übungstypen für verschiedene Lernstile, Web-Anwendung mit Vaadin statt Mobile-App, SQLite als eingebettete Datenbank.

2. Planung und Entwurf

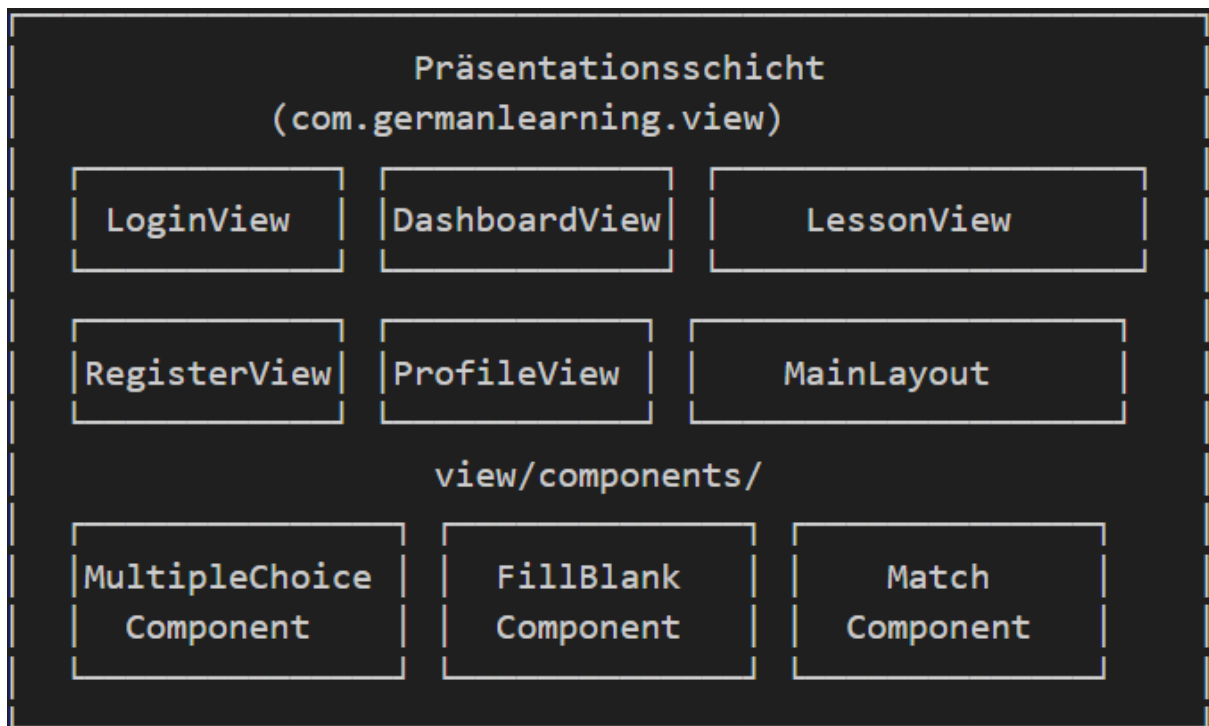
2.1 Technologien und Werkzeuge

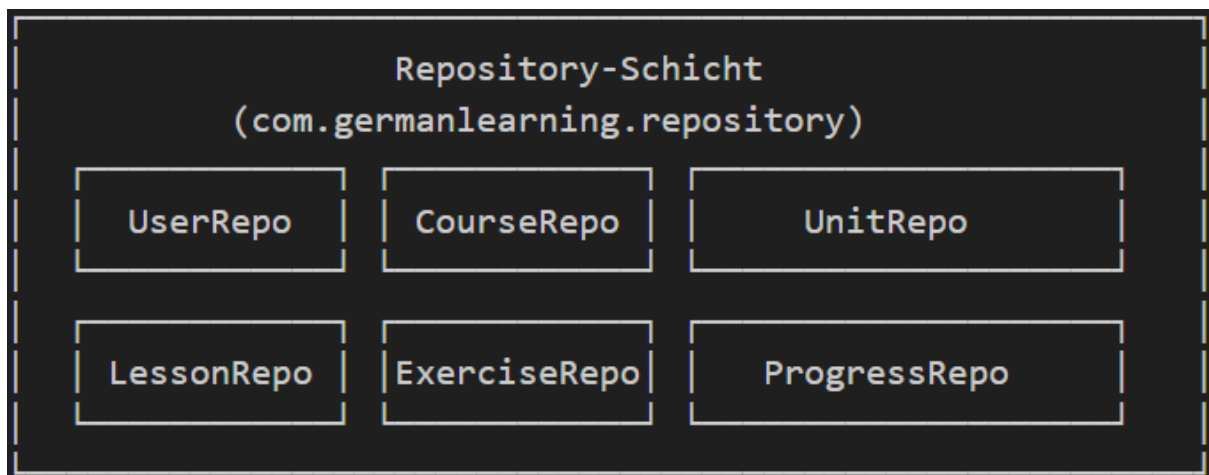
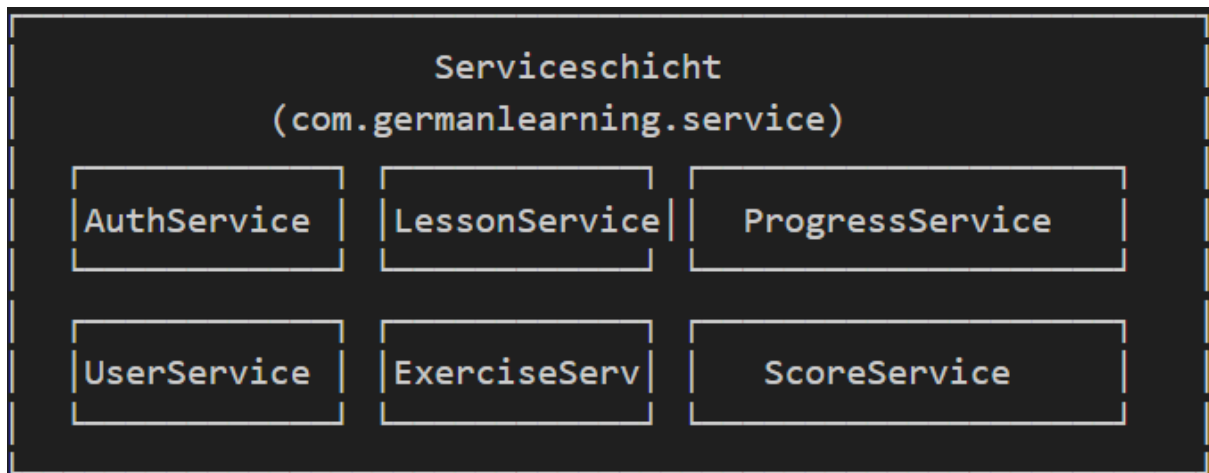
Die Anwendung basiert auf folgendem Java-Stack:

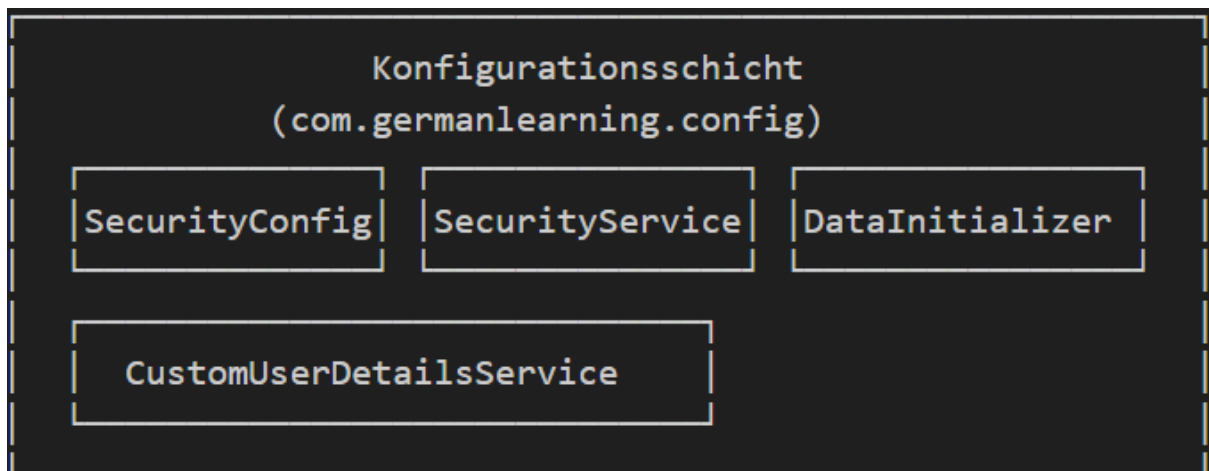
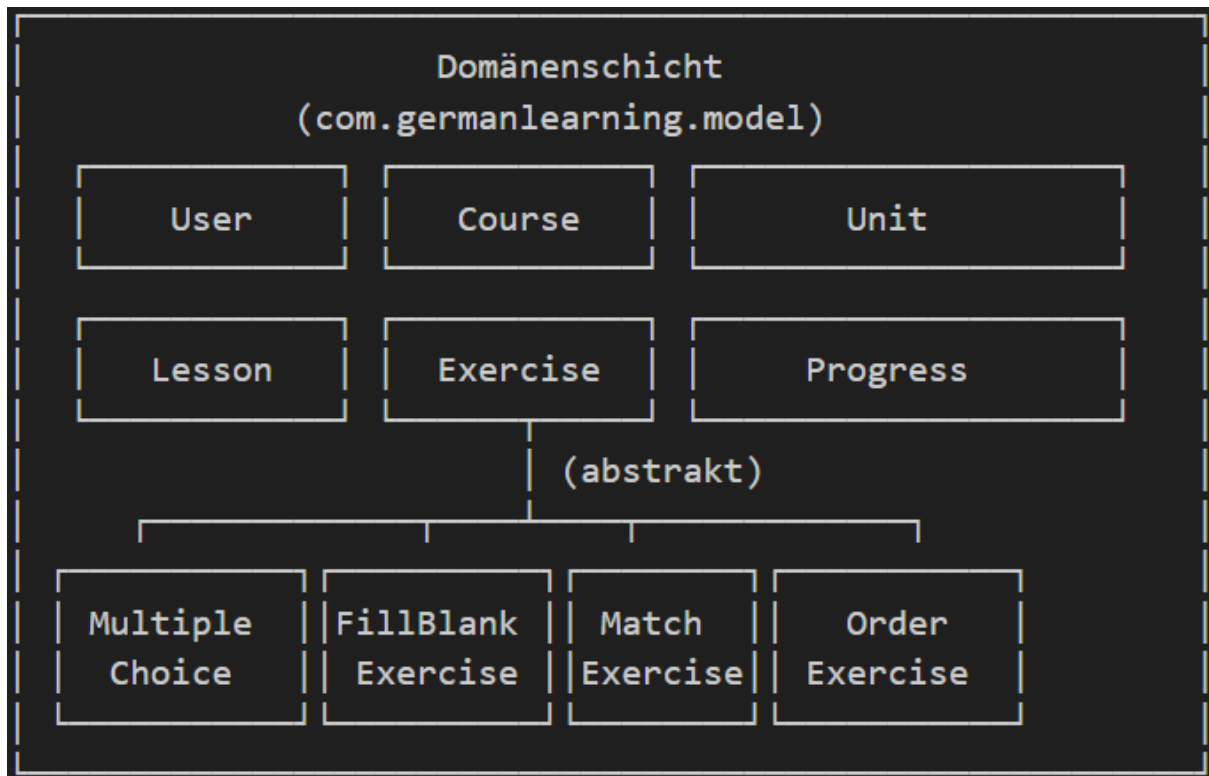
Technologie	Version	Verwendungszweck
Java	17 LTS	Programmiersprache
Spring Boot	3.2.0	Anwendungsframework mit Auto-Configuration
Vaadin	24.3.0	Serverseitiges Web-UI-Framework
SQLite	3.44.x	Eingebettete Datenbank
Hibernate	6.4.x	ORM mit hibernate-community-dialects
Maven	3.9	Build- und Dependency-Management
IntelliJ IDEA	2024.3	Entwicklungsumgebung

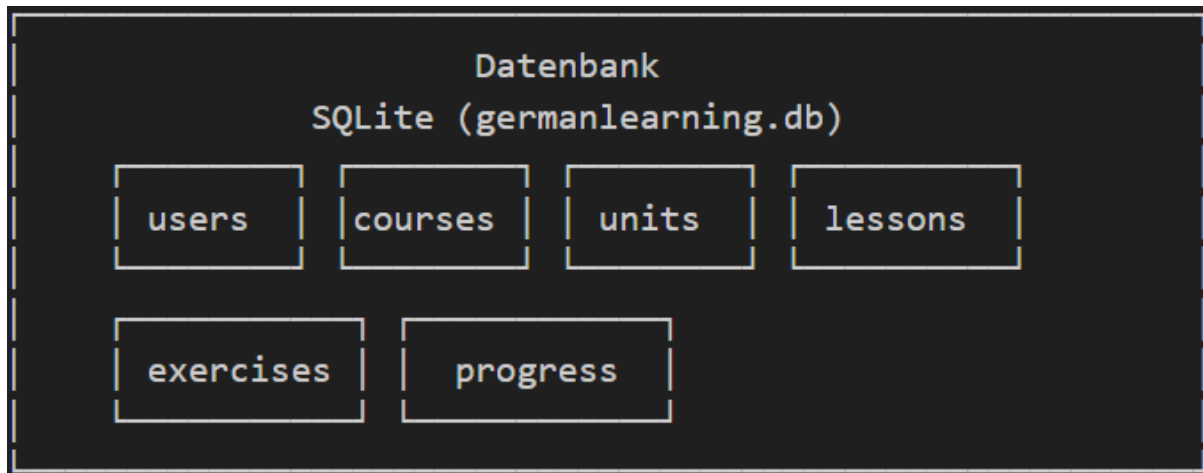
2.2 Architekturüberblick

Die Anwendung folgt einer klassischen Schichtenarchitektur:









Package-Struktur:

com.germanlearning/

└ GermanLearningApplication.java

Spring Boot Entry Point

└ config/

| └ SecurityConfig.java

Spring Security Konfiguration

| └ SecurityService.java

Authentifizierungs-Hilfsdienst

| └ CustomUserDetailsService.java

UserDetailsService für Spring Security

| └ DataInitializer.java

Initiale Testdaten (5 Lektionen, ~25 Übungen)

• Unit 1 "Basics": 3 Lektionen (Greetings, Introductions, Numbers)

• Unit 2 "Simple Sentences": 2 Lektionen (Articles, Verbs)

• Jede Lektion: 4-6 Übungen verschiedener Typen

└ model/

| └ User.java

Benutzer-Entität

• username, email, passwordHash

• totalXp, currentStreak, longestStreak

• createdAt, lastActiveAt

• @OneToMany progressList

└ Course.java	Kurs-Entität
└ Unit.java	Lerneinheit-Entität
└ Lesson.java	Lektion-Entität
└ Exercise.java	Abstrakte Übungs-Basisklasse
└ ExerciseType.java	Enum für Übungstypen
└ MultipleChoiceExercise.java	Multiple-Choice-Übung
└ FillBlankExercise.java	Lückentext-Übung
└ MatchExercise.java	Zuordnungs-Übung
└ OrderExercise.java	Satzordnungs-Übung
└─ Progress.java	Fortschritts-Entität
└ repository/	
└ UserRepository.java	
└ CourseRepository.java	
└ UnitRepository.java	
└ LessonRepository.java	
└ ExerciseRepository.java	
└─ ProgressRepository.java	
└ service/	
└ AuthService.java	Authentifizierungslogik
└ UserService.java	Benutzerverwaltung
└ LessonService.java	Lektions- und Freischaltlogik
└ ExerciseService.java	Übungsvalidierung
└ ProgressService.java	Fortschrittsverwaltung
└─ ScoreService.java	XP-Berechnung
└─ view/	
└ LoginView.java	Login-Seite
└ RegisterView.java	Registrierungsseite

└ DashboardView.java	Hauptübersicht
└ LessonView.java	Lektionsansicht
└ ProfileView.java	Benutzerprofil
└ MainLayout.java	Rahmen-Layout
└ components/	
└ ExerciseComponent.java	Abstrakte Übungskomponente
└ MultipleChoiceComponent.java	
└ FillBlankComponent.java	
└ MatchComponent.java	
└ OrderComponent.java	

2.3 Objektorientierter Entwurf

Klassenhierarchie und Vererbung

Die `Exercise`

-Klassenhierarchie demonstriert OOP-Konzepte:

...

Exercise (abstrakt)

└ MultipleChoiceExercise	(@DiscriminatorValue("MULTIPLE_CHOICE"))
└ FillBlankExercise	(@DiscriminatorValue("FILL_BLANK"))
└ MatchExercise	(@DiscriminatorValue("MATCH_PAIRS"))
└ OrderExercise	(@DiscriminatorValue("SENTENCE_ORDER"))

...

Die abstrakte Basisklasse definiert gemeinsame Attribute und die abstrakte Methode `validateAnswer()`, während Subklassen spezifische Validierungslogik implementieren.

Single Table Inheritance: Alle Übungstypen werden in einer einzigen Datenbanktabelle ``exercises`` gespeichert. Die Spalte ``discriminator_type`` unterscheidet die Subklassen. Zusätzlich existiert ein Enum-Feld ``exercise_type_enum`` für typsichere Abfragen im Code.

Beziehungen:

- Komposition (1:n mit orphanRemoval): Course → Unit → Lesson → Exercise
- Assoziation: Progress referenziert User und Lesson

Dependency Injection

```
```java
@Service
public class AuthService {
 private final UserRepository userRepository;
 private final PasswordEncoder passwordEncoder;

 public AuthService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
 this.userRepository = userRepository;
 this.passwordEncoder = passwordEncoder;
 }
}
```
```

3. Entwicklung

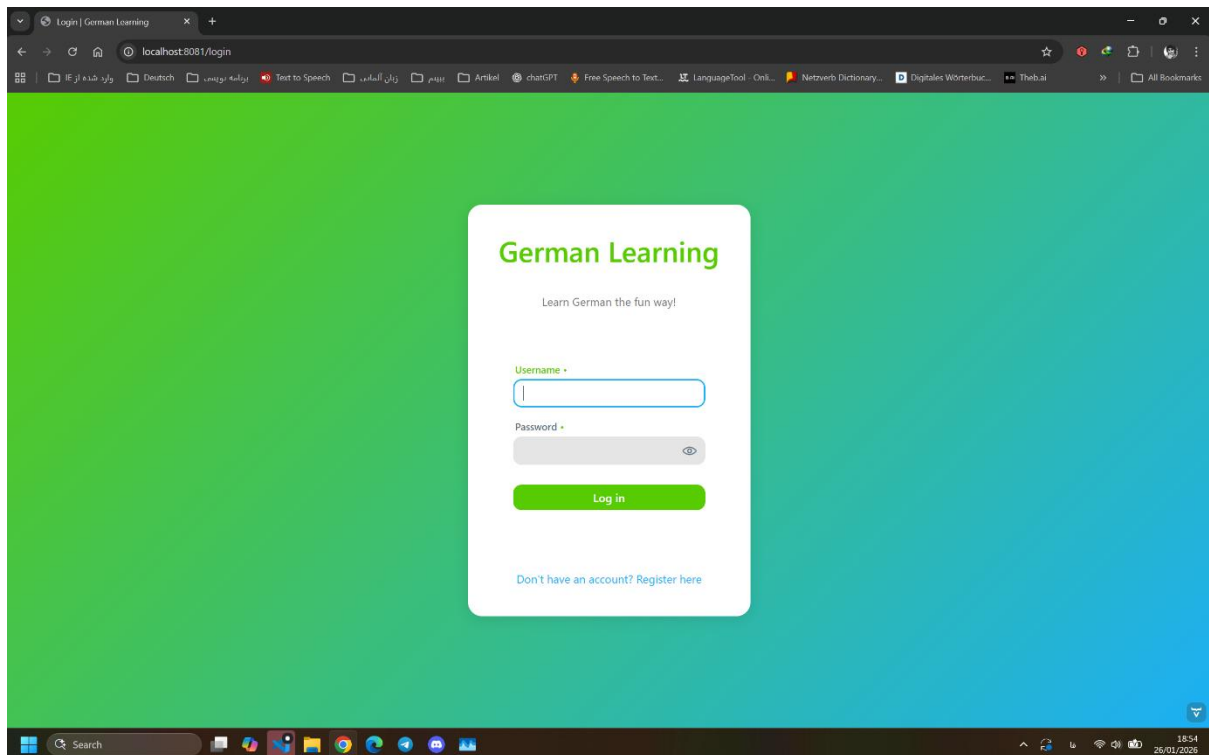
3.1 Implementierte Funktionen

Die Anwendung umfasst folgende implementierte Funktionen:

Authentifizierung:

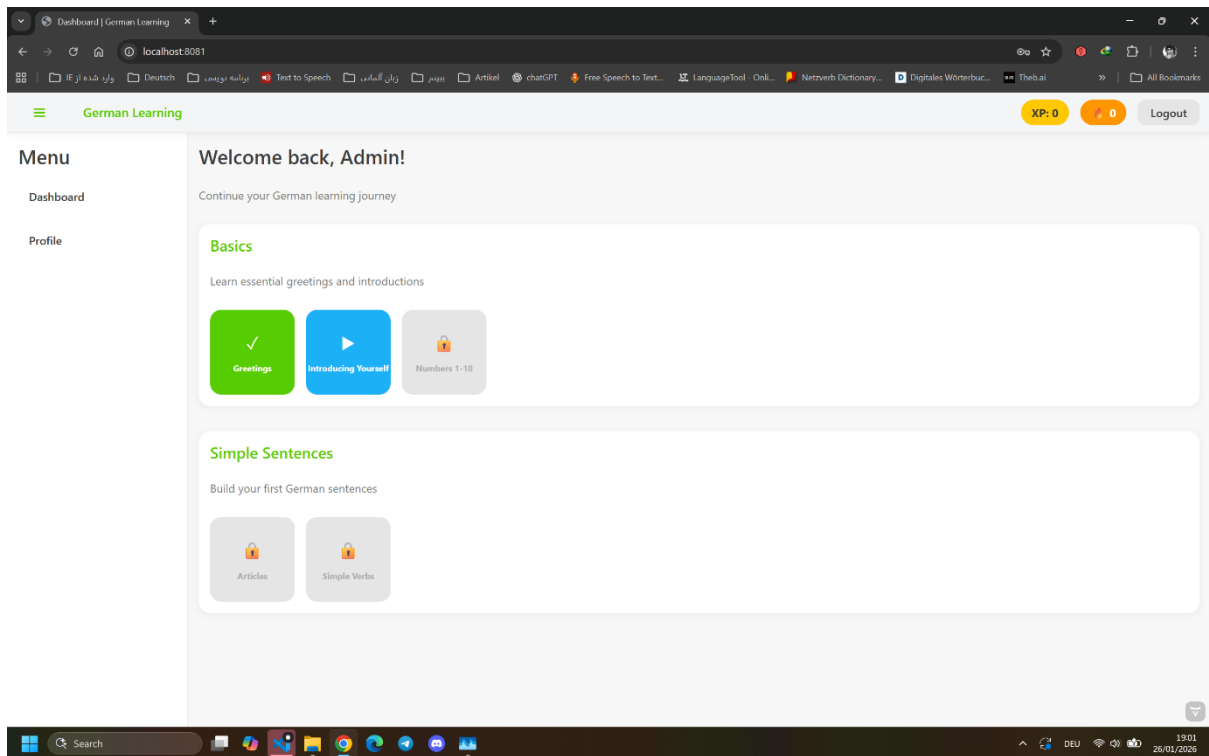
- Benutzerregistrierung mit Validierung (einzigartiger Benutzername/E-Mail)
- Login mit Benutzername und Passwort

- Automatische Session-Verwaltung
- Logout-Funktionalität



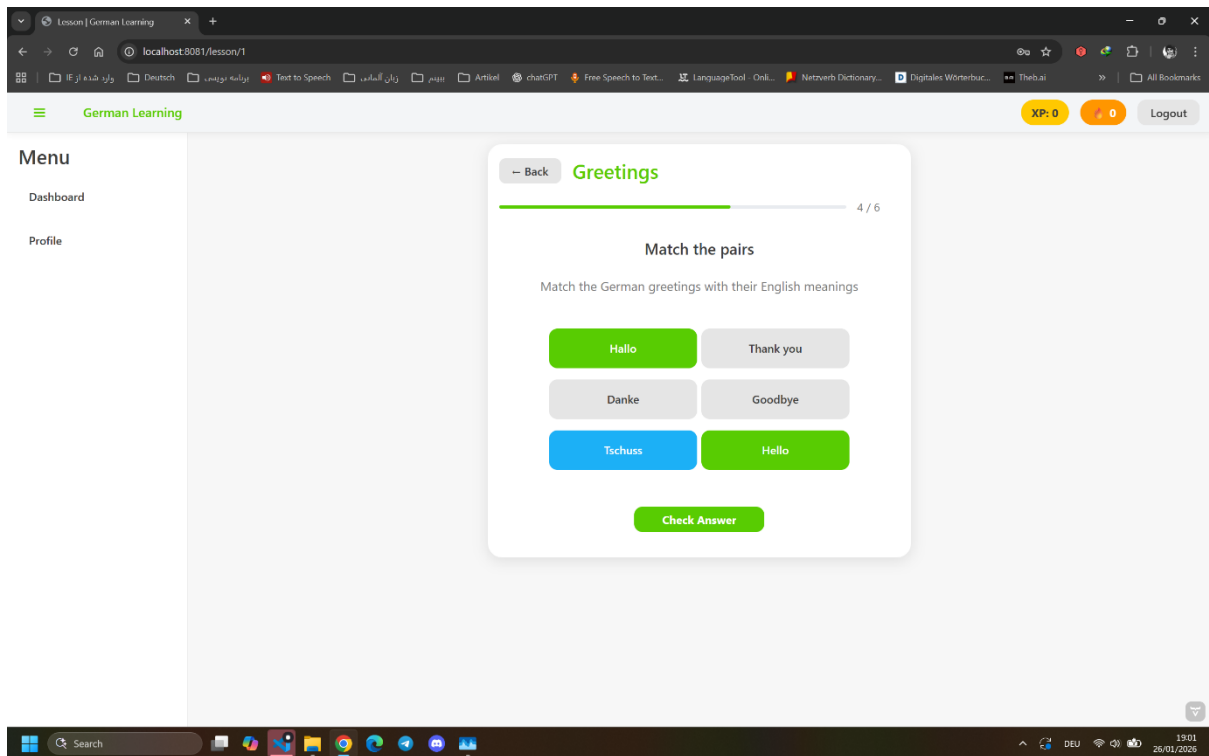
Dashboard:

- Übersicht aller Kurse und Units
- Visuelle Darstellung des Lektionsstatus (gesperrt/verfügbar/abgeschlossen)
- Navigation zu Lektionen



Lektionsansicht:

- Fortschrittsbalken zeigt aktuelle Position
- Anzeige der aktuellen Übung
- Navigation zwischen Übungen



Übungstypen:

Multiple Choice:

- Frage mit 4 Antwortoptionen
- Radio-Button-Auswahl
- Sofortige Validierung

Fill Blank (Lückentext):

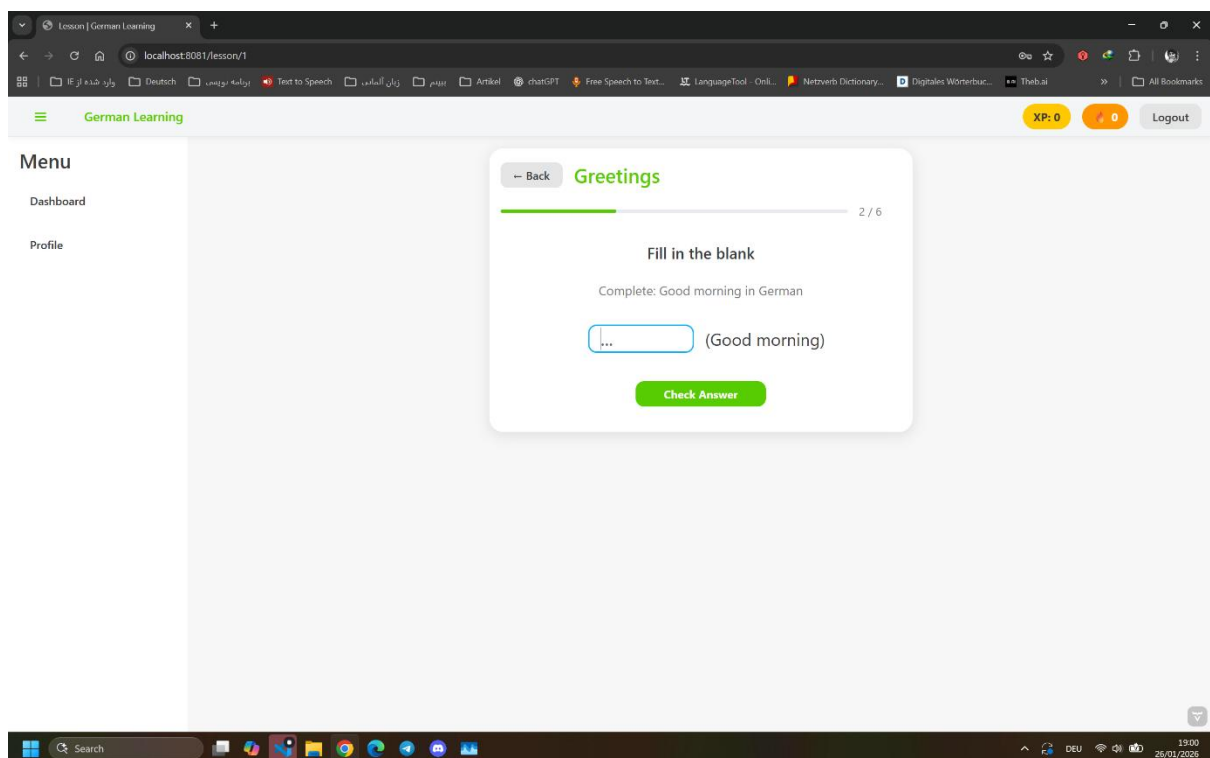
- Satz mit Lücke (gespeichert in ``sentenceTemplate``)
- Texteingabefeld
- Unterstützung alternativer korrekter Antworten (durch Pipe-Separator: "Antwort1|Antwort2")
- Case-insensitive Validierung mit Normalisierung

Match Pairs (Zuordnung):

- Deutsche Begriffe mit englischen Übersetzungen verbinden
- Datenformat: "Key1:Value1;Key2:Value2;Key3:Value3" (in ``pairsData`` ``pairsData``)
- Validierung: Alle Paare müssen korrekt zugeordnet sein
- Flexible Map-basierte Implementierung mit ``getPairs()`` und ``setPairs(Map)``

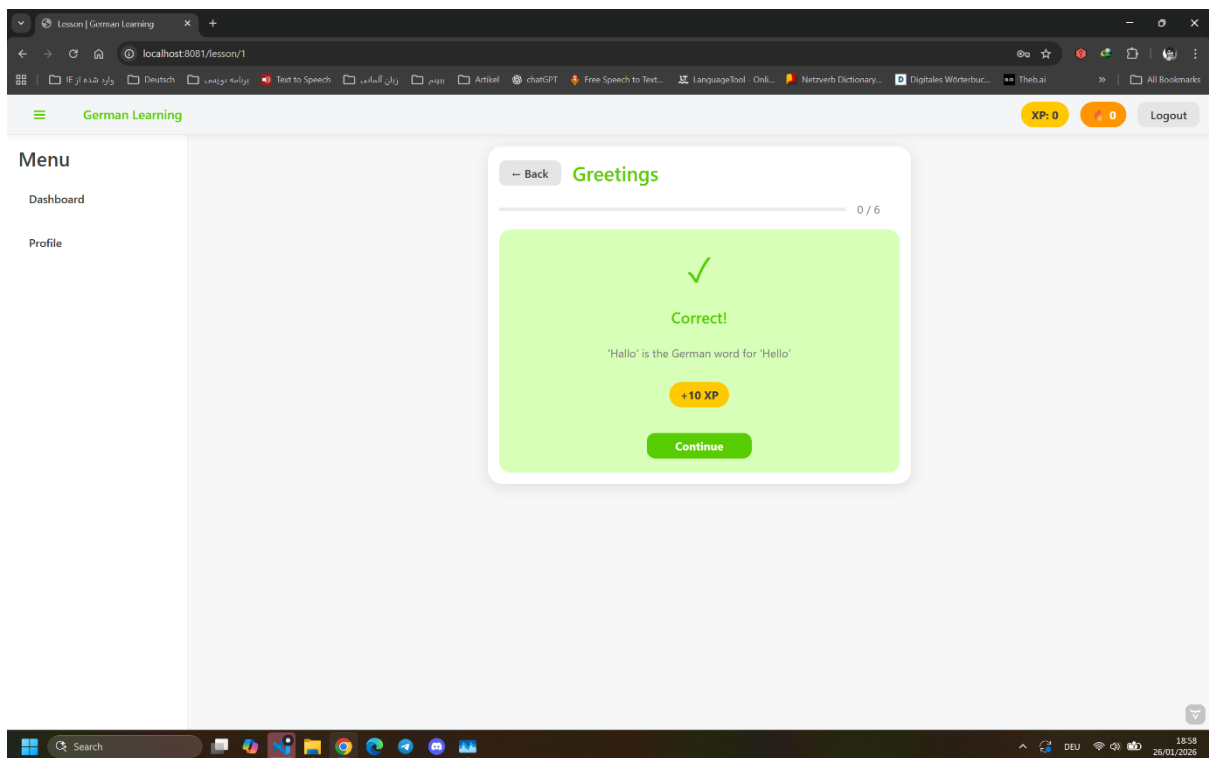
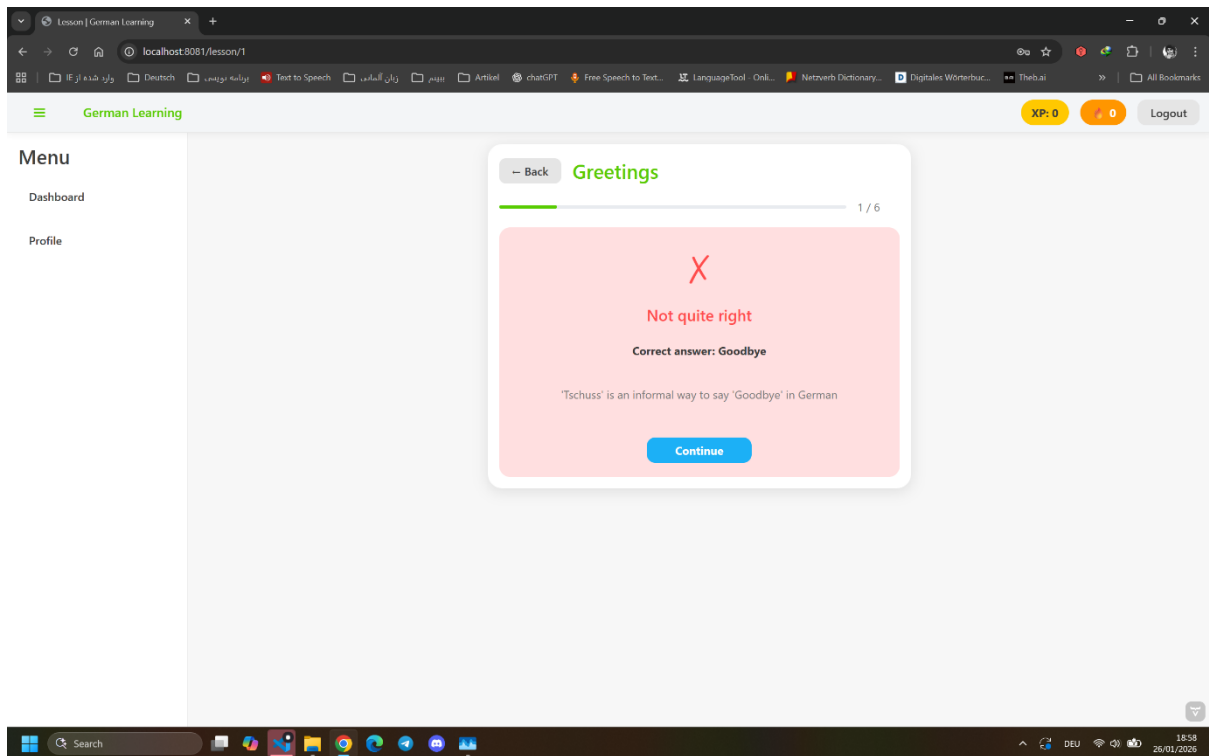
Sentence Order (Satzordnung):

- Durchmischte Wörter (gespeichert in ``shuffledWords`` als "Wort1|Wort2|Wort3")
- Korrekte Reihenfolge in ``correctOrder`` (als vollständiger Satz)
- Validierung mit Normalisierung (Whitespace, Case-insensitive)
- Hilfsmethode ``getShuffledWordsList()`` für UI-Darstellung



Feedback-System:

- Grüne Anzeige bei korrekter Antwort mit XP-Badge
- Rote Anzeige bei falscher Antwort mit korrekter Lösung
- Erklärungstext zur Übung

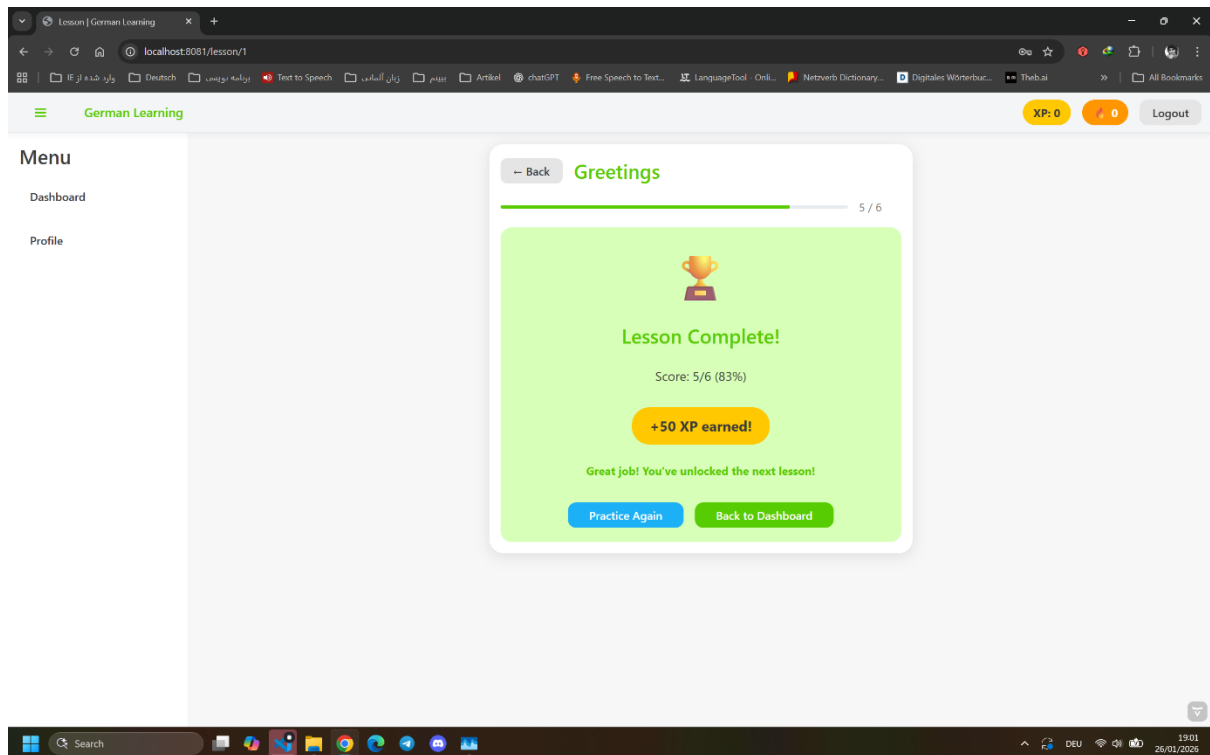


Abschluss-Summary:

- Gesamtpunktzahl nach Lektionsabschluss
- Anzahl korrekter Antworten

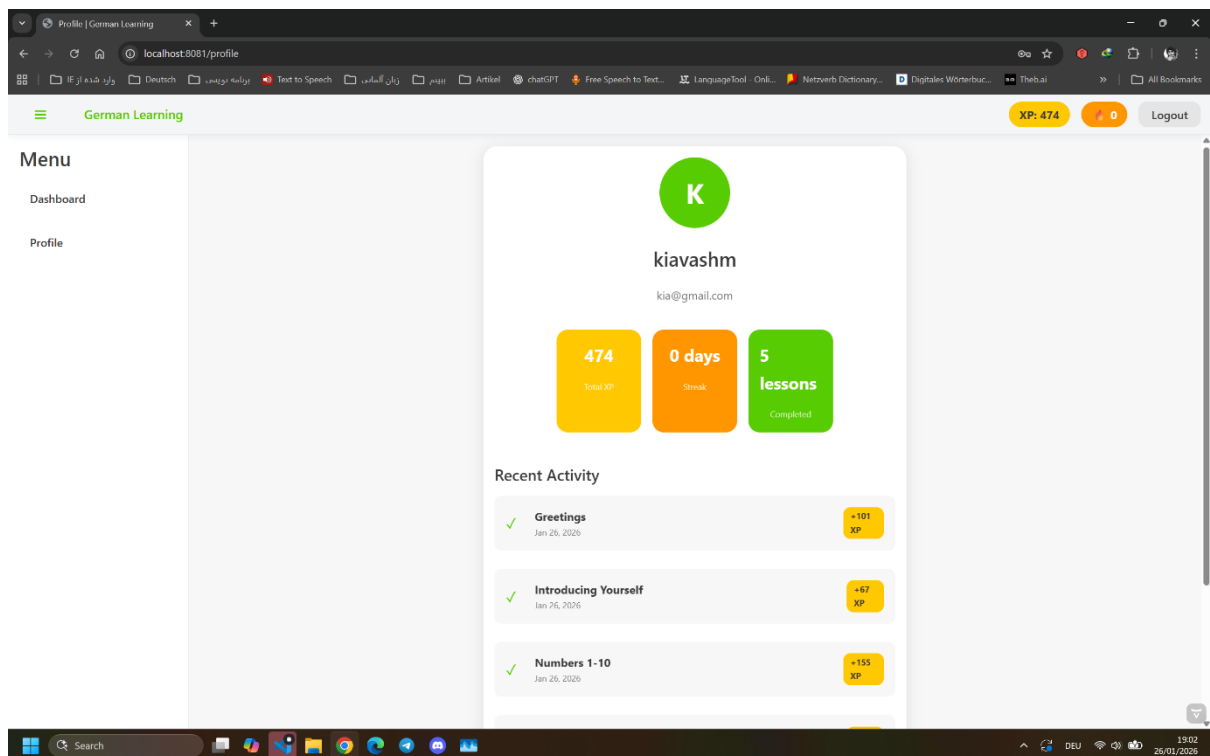
- Verdiente XP

- Bestanden/Nicht bestanden-Anzeige



Benutzerprofil:

- Avatar mit Initialen
- Gesamte XP-Punkte
- Aktueller Streak
- Abgeschlossene Lektionen
- Lernhistorie



3.2 OOP-Konzepte im Projekt

Vererbung und Polymorphie

Die `Exercise`-Klassenhierarchie nutzt Polymorphie für einheitliche Behandlung verschiedener Übungstypen:

```
```java
@Entity
@Table(name = "exercises")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "discriminator_type", discriminatorType =
DiscriminatorType.STRING)
public abstract class Exercise {
 @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

 @NotBlank
 @Column(nullable = false, length = 1000)
 private String question;

 @NotBlank
 @Column(nullable = false, length = 500)
```

```

private String correctAnswer;

@Column(length = 1000)
private String explanation; // Erklärung zur Übung

@Column(nullable = false)
private int xpReward = 10; // XP-Punkte für korrekte Antwort

@Enumerated(EnumType.STRING)
@Column(name = "exercise_type_enum", nullable = false)
private ExerciseType exerciseType; // Übungstyp als Enum

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "lesson_id", nullable = false)
private Lesson lesson;

public abstract boolean validateAnswer(String userAnswer);
}

@Entity
@DiscriminatorValue("MULTIPLE_CHOICE")
public class MultipleChoiceExercise extends Exercise {
 @Column(length = 1000)
 private String options; // Pipe-separated:
 "Option1|Option2|Option3|Option4"

 @Column
 private int correctOptionIndex; // 0-basierter Index

 @Override
 public boolean validateAnswer(String userAnswer) {
 try {
 int selectedIndex = Integer.parseInt(userAnswer.trim());
 return selectedIndex == correctOptionIndex;
 } catch (NumberFormatException e) {
 // Fallback: Textvergleich mit correctAnswer
 return
userAnswer.trim().equalsIgnoreCase(getCorrectAnswer().trim());
 }
 }

 // Hilfsmethoden für Array-Konvertierung
 public String[] getOptionsArray() {
 return options != null ? options.split("\\|") : new String[0];
 }
}
...

```

In der `LessonView` wird Polymorphie genutzt:

```
```java
private ExerciseComponent createExerciseComponent(Exercise exercise) {
    if (exercise instanceof MultipleChoiceExercise) {
        return new MultipleChoiceComponent((MultipleChoiceExercise) exercise);
    } else if (exercise instanceof FillBlankExercise) {
        return new FillBlankComponent((FillBlankExercise) exercise);
    }
    // ... weitere Typen
}

// Validierung nutzt ebenfalls Polymorphie
boolean isCorrect = exercise.validateAnswer(userAnswer); // Ruft spezifische
Implementierung auf
```
```

Kapselung

```
```java
@Entity
@Table(name = "progress")
public class Progress {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "lesson_id", nullable = false)
    private Lesson lesson;

    private int correctAnswers = 0;
    private int totalAnswers = 0;
    private int attempts = 0;           // Anzahl der Versuche
    private boolean completed = false;  // Abschlussstatus
    private int xpEarned = 0;           // Verdiente XP
    private boolean xpLocked = false;   // XP-Sperre (keine erneute Vergabe)
    private double bestScore = 0.0;     // Beste Punktzahl

    private LocalDateTime completedAt;
    private LocalDateTime lastAttemptAt;

    public void incrementCorrectAnswers() {
        this.correctAnswers++;
    }
}
```

```

    public void incrementTotalAnswers() {
        this.totalAnswers++;
    }

    public double getScorePercentage() {
        if (totalAnswers == 0) return 0.0;
        return (double) correctAnswers / totalAnswers * 100.0;
    }
}
...

```

3.3 Technische Herausforderungen & Lösungen

SQLite mit Hibernate

Problem: Hibernate unterstützt SQLite nicht standardmäßig. Zusätzliche Herausforderungen bei der Entwicklung auf Cloud-Plattformen wie Replit.

Lösung: Integration von `hibernate-community-dialects` und spezielle Konfiguration für SQLite-Kompatibilität:

```
```properties
```

```
SQLite Dialect
```

```
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
```

```
Kompatibilitäts-Fixes (insbesondere für Replit)
```

```
spring.jpa.properties.hibernate.jdbc.use_get_generated_keys=false
```

```
spring.jpa.properties.hibernate.id.db_structure_naming_strategy=single
```

```
```
```

Diese Konfigurationen verhindern Probleme mit der automatischen ID-Generierung in SQLite.

Vaadin-Spring Security Integration

Problem: Standard-Spring-Security blockierte Vaadin-Ressourcen.

Lösung: `VaadinWebSecurity` als Basisklasse statt direkter `SecurityFilterChain` - Konfiguration:

```
```java
@Configuration
@EnableWebSecurity
public class SecurityConfig extends VaadinWebSecurity {

 @Override
 protected void configure(HttpSecurity http) throws Exception {
 http.authorizeHttpRequests(auth -> auth
 .requestMatchers(new
AntPathRequestMatcher("/register")).permitAll()
 .requestMatchers(new AntPathRequestMatcher("/login")).permitAll()
 .requestMatchers(new
AntPathRequestMatcher("/images/**")).permitAll()
 .requestMatchers(new
AntPathRequestMatcher("/icons/**")).permitAll()
);

 super.configure(http); // Vaadin-spezifische Security
 setLoginView(http, LoginView.class);
 }
}
```
```

XP-Doppelvergabe verhindern

Problem: Bei Wiederholung sollten keine zusätzlichen XP vergeben werden.

Lösung: `xpLocked` -Flag in Progress-Entität:

```
```java
if (!progress.isXpLocked()) {
 // XP vergeben
}
progress.setXpLocked(true); // Nach Bestehen
```
```

4. Inbetriebnahme

4.1 Voraussetzungen

- Java JDK 17+ (OpenJDK oder Oracle JDK)
- Maven 3.9+
- Aktueller Webbrowser (Chrome, Firefox, Edge)

4.2 Lokale Inbetriebnahme

```
```bash
 1. In Projektverzeichnis wechseln
cd OOSD

 2. Abhängigkeiten laden und bauen
mvn clean install

 3. Anwendung starten
mvn spring-boot:run

 4. Browser öffnen
http://localhost:8081
```
```

Nutzung:

1. Registrierung mit Benutzernamen, E-Mail und Passwort
 2. Login und Dashboard mit Lektionsübersicht
 3. Übungen durcharbeiten, XP sammeln
 4. Profil mit Statistiken anzeigen
-

5. Fazit

5.1 Anforderungsabgleich

| Anforderung | Status |
|--------------------------------|-----------|
| ----- | ----- |
| FA-01: Benutzerregistrierung | ✓ Erfüllt |
| FA-02: Benutzeranmeldung | ✓ Erfüllt |
| FA-03: Kursstruktur navigieren | ✓ Erfüllt |
| FA-04: Lektionen absolvieren | ✓ Erfüllt |
| FA-05: 4 Übungstypen | ✓ Erfüllt |
| FA-06: Feedback | ✓ Erfüllt |
| FA-07: XP-System | ✓ Erfüllt |
| FA-08: Fortschrittsverfolgung | ✓ Erfüllt |
| FA-09: Freischaltlogik | ✓ Erfüllt |
| FA-10: Übungsmodus | ✓ Erfüllt |
| FA-11: Benutzerprofil | ✓ Erfüllt |

5.2 Lernerfahrungen

Technische Erkenntnisse:

- Vaadin's serverseitiger Ansatz vereinfacht UI-Entwicklung, erfordert aber Umdenken bei Skalierbarkeit
- SQLite mit Hibernate erfordert spezielle Dialect-Konfigurationen
- Spring Security mit Vaadin funktioniert gut über `VaadinWebSecurity``

Methodische Erkenntnisse:

- Früher Fokus auf Domänenmodell zahlte sich aus
- Iterative Entwicklung ermöglichte kontinuierliches Testing

5.3 Erweiterungsmöglichkeiten

Priorität 1: Automatisierte Tests, Responsive Design, mehr Inhalte

Priorität 2: Spaced Repetition, Audio-Übungen, Leaderboard

Priorität 3: REST-API, OAuth2-Login, Admin-Panel

Priorisierte Erweiterungen (Ausblick)

Die folgenden Erweiterungen können in Zukunft implementiert werden, um die Anwendung zu verbessern:

Priorität 1 – Kurzfristig:

1. Automatisierte Tests: Unit-Tests für Services, Integration-Tests für Repositories
2. Responsive Design: Anpassung für mobile Geräte
3. Mehr Inhalte: Zusätzliche Lektionen und Übungen für A1-Niveau

Priorität 2 – Mittelfristig:

4. Spaced Repetition: Wiederholungssystem basierend auf Vergessenskurve
5. Audio-Übungen: Hörverständnis mit Audio-Dateien
6. Leaderboard: Rangliste der Benutzer nach XP

Priorität 3 – Langfristig:

7. REST-API: Für potenzielle Mobile-App oder externe Integration
8. OAuth2-Login: Anmeldung über Google/GitHub
9. Mehrsprachigkeit: Weitere Sprachen neben Deutsch

6. Quellen

Frameworks und Bibliotheken

- Spring Boot (Version 3.2.0) – Anwendungsframework, Dependency Injection, eingebetteter Web-Server.
- Spring Security (Version 6.2.x, via Spring Boot) – Authentifizierung und Autorisierung.
- Spring Data JPA (Version 3.2.x, via Spring Boot) – Repository-Abstraktion für Datenbankzugriff.
- Vaadin (Version 24.3.0) – Serverseitiges Web-UI-Framework für Java.
- Hibernate ORM (Version 6.4.x, via Spring Boot) – JPA-Implementierung, Object-Relational Mapping.
- Hibernate Community Dialects (Version 6.4.x) – SQLite-Unterstützung für Hibernate.
- SQLite JDBC (Version 3.44.1.0) – JDBC-Treiber für SQLite-Datenbank.

| Technologie | Beschreibung |
|--|----------------------|
| Spring Boot 3.2.0 | Anwendungsframework |
| Spring Security 6.2.x | Authentifizierung |
| Vaadin 24.3.0 | Web-UI-Framework |
| Hibernate ORM 6.4.x | JPA-Implementierung |
| hibernate-community-dialects | SQLite-Unterstützung |
| SQLite JDBC 3.44.1.0 | Datenbank-Treiber |
| IntelliJ IDEA 2024.3 | Entwicklungsumgebung |
| Apache Maven 3.9.x | Build-Management |
| - Eine kontinuierliche Integration (CI) hätte Build-Probleme früher aufgedeckt | |

Entwicklungsumgebung

- IntelliJ IDEA (Version 2024.3) – Integrierte Entwicklungsumgebung für Java.

- Apache Maven (Version 3.9.x) – Build-Management und Dependency-Management.